



PLANT-DISEASE DETECTION LAB

ClearML MLOps

Hands-On

The full MLOps loop in 4 hours — track, scale, compare, and reuse a Vision Transformer, running on your own H200 cluster.



202.131.110.56



Self-hosted OSS



Browser + Colab only

Your four hours

One MLOps loop, four moves. Everything runs in your browser — notebooks in Google Colab, training on the H200s, results live in the ClearML UI.



01

HOUR 1

Track

Log your first experiment. Every run's code, params, plots & data captured automatically.

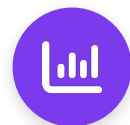
 Colab (CPU)

02

HOUR 2

Scale

Submit training to a shared GPU queue. Watch agents run it on MIG-sliced H200s.

 H200 agent

03

HOUR 3

Compare

Clone, tweak, and compare runs. Pick the winner by validation accuracy.

 Colab + UI

04

HOUR 4

Reuse

Publish the winner to the registry, pull it back by tag, and run inference.

 Colab (CPU)

How today works

A teaching lab, not a Kaggle contest — small model, tiny dataset, fast runs, so every ClearML concept is visible.



All you need is a browser

A laptop and a Google account. Notebooks open in Colab; nothing heavy installs on your machine.



One shared server, your own space

Everyone connects to the same ClearML server with a shared key; your runs land in your own project by name.



Small on purpose

ViT-Tiny (~5.7M params), ~1000 images, 2-3 epochs — runs finish in seconds so we watch the workflow, not a progress bar.

THE GOAL

Learn ClearML, not build a production model.

By 4:00 you'll have run the entire loop end-to-end:

track → scale → compare → register → reuse

~4 hours

Machine learning without MLOps

Sound familiar? These are the exact gaps ClearML closes — one per hour today.



“Which run was that?”

Results in notebook cells and screenshots. No record of the code, params, or data behind a number.



GPUs are a free-for-all

People SSH in, clobber each other, run out of memory. No queue, no fair sharing, no isolation.



Comparing is manual

Copy-pasting metrics into a spreadsheet to figure out which hyperparameters actually helped.



Models get lost

“final_v3_really.pt” on someone's laptop. No lineage, no tags, no way to reproduce it.

What ClearML gives you

An open-source MLOps platform. Five capabilities — and today maps almost one-to-one onto them.



Experiment Tracking

Auto-log code, params, metrics, plots

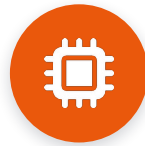
Hour 1



Data Versioning

Immutable, versioned datasets

Hour 1



Orchestration

Queues + agents run jobs on GPUs

Hour 2



Model Registry

Catalog, tag, publish, reuse models

Hour 3-4



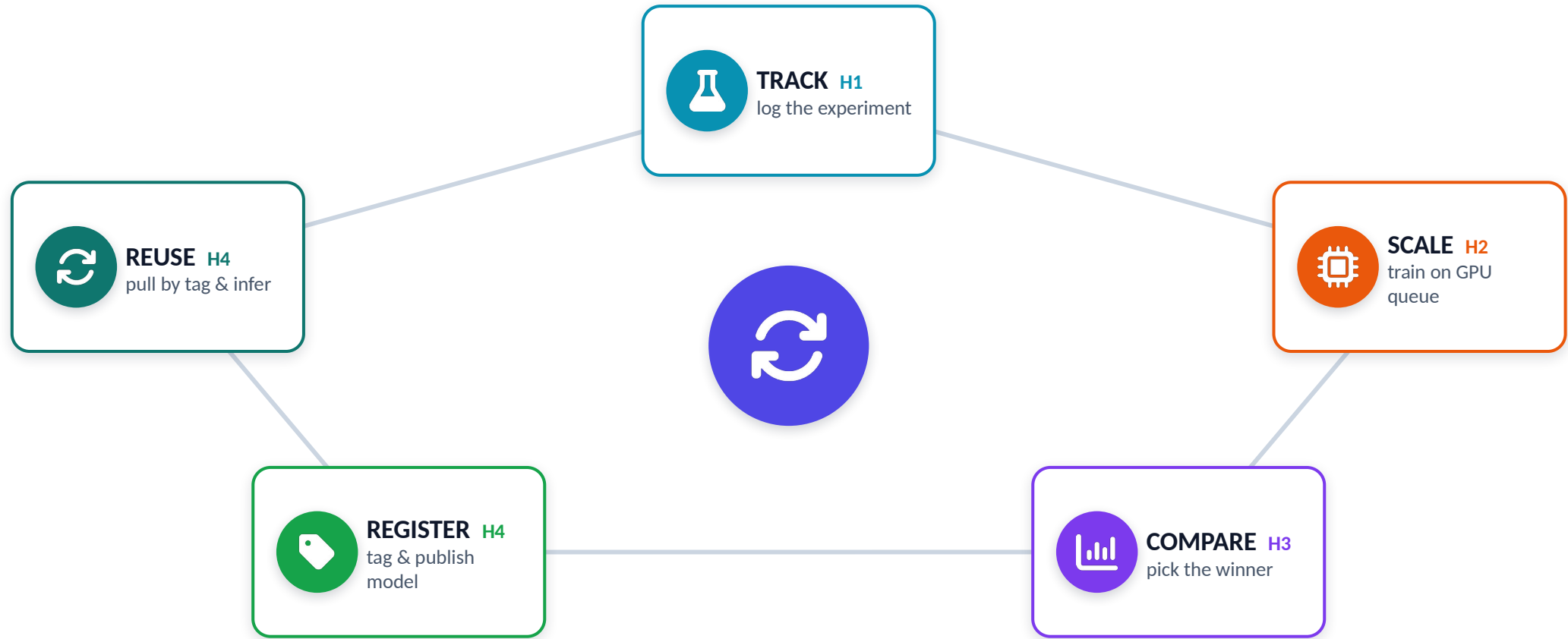
Serving

Deploy models as live endpoints

What's next

The MLOps loop — the whole day on one slide

Each step is a ClearML capability and a specific hour. The loop closes: a model produced by one run is reused by another.



What's deployed for you

This isn't a sandbox. Today runs on a real, self-hosted ClearML server and a shared dual-H200 HPC box.



2× H200

NVIDIA NVL GPUs, ~141 GB VRAM
each



344

logical CPU cores



~1 TB

system RAM



21 TB

RAID storage for datasets & artifacts



ClearML server <http://202.131.110.56>

Web UI on port 80 · plain HTTP is fine on this internal box · one login shared by the room



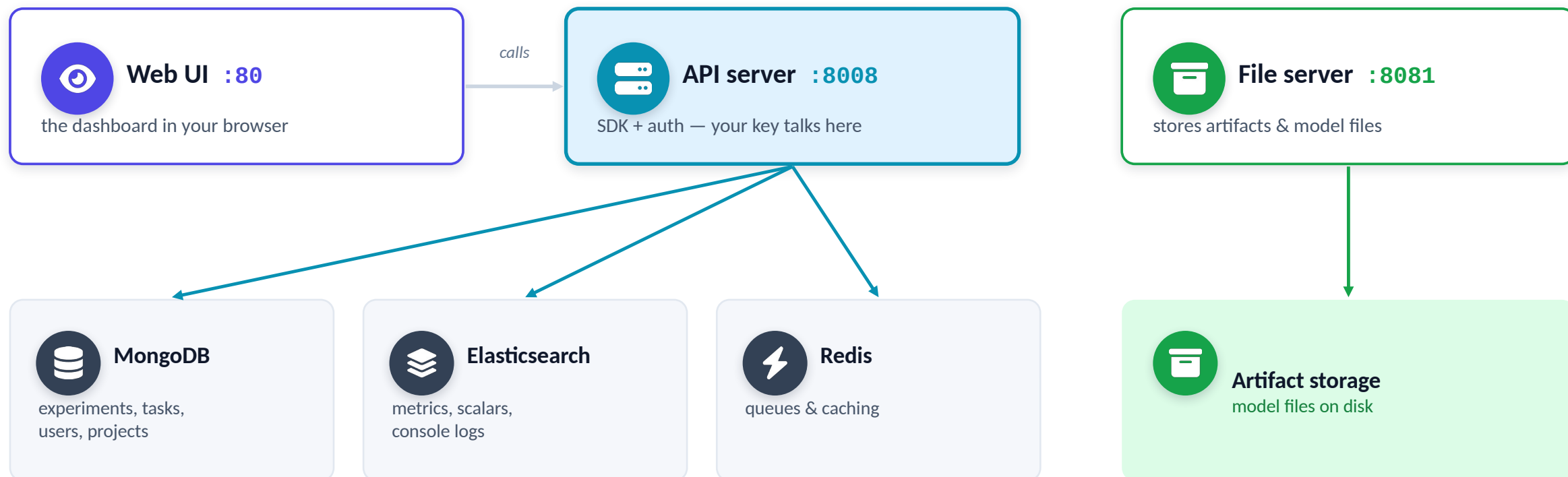
Docker + NVIDIA runtime



Open-source install

The ClearML server, under the hood

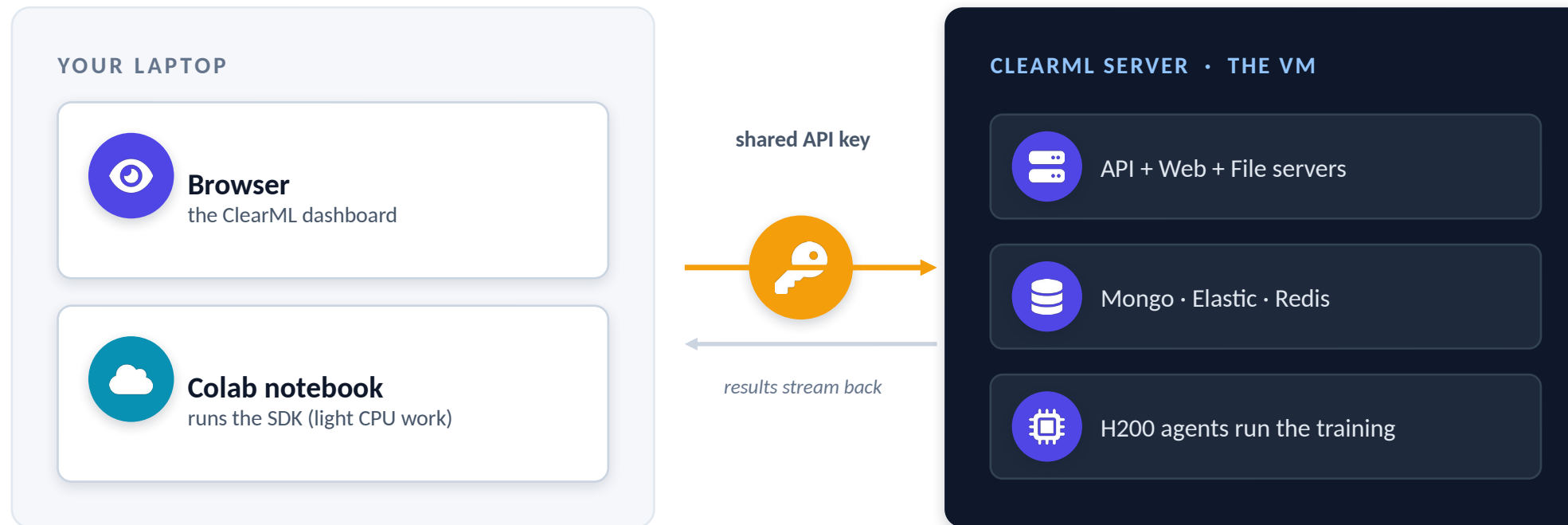
A stack of Docker containers. The API server is the hub — it reads and writes all three data stores.



Backing stores — you never connect to these directly; the API server does.

What actually happens on your laptop

You never install ClearML's heavy stack. Your browser and a Colab notebook talk to the server over your API key.



Nothing heavy runs on your laptop — the browser shows the UI, Colab submits work, the VM does the rest.



HOUR 1 • TRACK

Track

Log in, run your first tracked experiment, and watch ClearML capture everything automatically.

In this hour

- Projects & Tasks — the two core objects
- Auth & API keys — how the OSS server keeps you apart
- Task.init() — automatic tracking in two lines
- ClearML Datasets + live EDA in the UI

~50 min



Projects & Tasks — the two objects you'll live in



A Task = one run

The atom of ClearML. It bundles the code, packages, params, metrics, plots, console log and output models of a single execution — with its own URL.

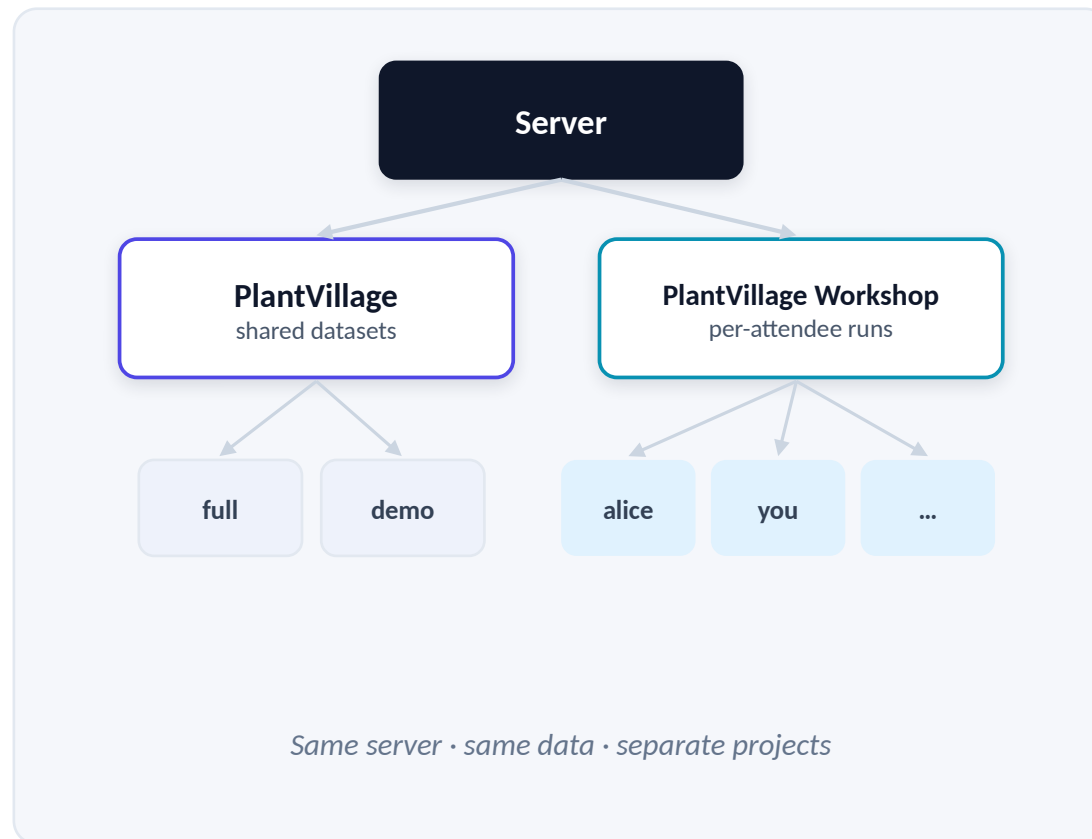


A Project = a folder of tasks

Groups related runs. Sub-projects nest with “/”. Datasets live in their own project too.



You share the server — not your work. Your runs land in **PlantVillage Workshop/<your-name>**.





Logging in: how the open-source server handles access

The OSS edition keeps auth deliberately simple. Know these three facts before you paste your key.



One shared login for the room

Attendees don't get individual accounts — everyone signs in as the same “workshop” user with one shared API key from the facilitator.



No RBAC in open-source ClearML

There are no per-user roles or permissions. Anyone signed in can see everyone's projects — good to know, don't put secrets in run names.



You're kept apart by project name

Setting `WORKSHOP_USER` routes your runs to your own sub-project, so 30 people share the server without stepping on each other.



One cell connects your notebook to the server

Paste the shared key and your name. The SDK reads these env-vars and talks to the API server on :8008.

credentials cell

```
import os
os.environ['CLEARML_API_HOST']      = '...:8008'
os.environ['CLEARML_FILES_HOST']    = '...:8081'
os.environ['CLEARML_API_ACCESS_KEY'] = '<shared>'
os.environ['CLEARML_API_SECRET_KEY'] = '<shared>'
os.environ['WORKSHOP_USER']         = 'alice'

# your runs → PlantVillage Workshop/alice
```



The key is your identity

No username/password in code — the access + secret key authenticate every SDK call.



Set with `os.environ`

Plain env-vars (not `%env`) so they're inherited by the `!python` training calls in later hours.



`WORKSHOP_USER = you`

The one line that routes everything you do into your own project space.

Task.init() — tracking in two lines

The moment this runs, ClearML starts capturing. You write ML code as usual; tracking happens underneath.

```
from clearml import Task
task = Task.init(
    project_name=PROJECT, task_name='EDA')
```

the two lines



Zero extra code

You keep writing normal training code — ClearML captures all of it in the background.

Captured automatically, the moment it runs:



Source code



Notebook cells



Installed packages



Environment



Console output



Params & metrics



Plots



Output models



Demo — your first tracked run

1

Paste the shared key + your name

Run the credentials cell (cells 0–1 in the notebook).

2

Run `Task.init()`

Two lines open the experiment and print your task's URL.

3

Open the task page

Click the printed link → your run in the ClearML UI.



Watch for



Your project appears: PlantVillage Workshop/<you>



The EXECUTION tab already lists your packages & code



It works from Colab with zero local install



Nothing to configure — init did it all



ClearML Datasets — versioned, shared data

Data gets the same treatment as code: versioned and immutable, so every run records exactly what it trained on.



Registered once, pulled anywhere

`Dataset.get(...).get_local_copy()` downloads and caches the files — you reference a version, not a path on someone's disk.



Immutable & versioned

A finalized dataset can't change. New data = a new version, so results stay reproducible.



One shared copy for the room

Everyone pulls the same `plantdisease_demo` (~1000 imgs, 4 classes); the full ~20k-image catalog sits alongside it.

plantdisease_full

~20.6k images • 15 classes

the catalog (browse only)

plantdisease_demo

~1000 images • 4 classes

what we actually train on today



The Datasets panel

Every dataset is a first-class object in the UI — with the same lineage and versioning you get for experiments.



Versions

Each finalized version has its own ID and can be fetched by name or version.



Lineage

See which task consumed which dataset version — data provenance for every run.



Contents & preview

Browse the file list, class folders, and sample previews without downloading.



Pull on demand

`get_local_copy()` materializes files locally and caches them for reuse.



Demo — EDA that logs itself

We explore the dataset — and every figure lands in the UI the moment we render it. No manual uploading.



Class distribution

A seaborn bar chart of images-per-class → auto-captured on `plt.show()`.

→ PLOTS tab



Sample-leaf grid

A grid of example leaves reported as a PNG → shows under DEBUG SAMPLES.

→ DEBUG SAMPLES tab



Single values

`report_single_value(total_images, num_classes)` → the summary section.

→ SCALARS tab



The anatomy of a task

After one run, click through these tabs — this is where you'll spend the rest of the day.



EXECUTION

code, git/uncommitted changes, packages, base image



CONFIGURATION

hyperparameters — editable when you clone (Hour 3)



SCALARS

loss, accuracy, GPU utilization over time



PLOTS

interactive charts (bar, confusion matrix, ...)



DEBUG SAMPLES

images & media — the leaf grids, predictions



ARTIFACTS

output models & files produced by the run



Hour 1 — what you can now do

You have a tracked experiment with plots, data lineage, and full environment capture — from a browser.



Connected a notebook to the server with a shared key



Understood projects vs tasks, and the OSS no-RBAC model



Ran Task.init() and saw automatic capture



Explored a versioned dataset and logged EDA to the UI



Next: Hour 2

So far everything ran in Colab on CPU. Next we send real training to the H200 GPU queue — and watch the whole room's jobs share the card.

Scale it up →



HOURL 2 • SCALE

Scale

Stop training on your laptop. Submit the job to a shared GPU queue and let agents run it on the H200s.

In this hour

- Queues, workers & agents — the orchestration model
- Fractional GPUs & MIG — how one H200 is shared
- How your code + deps reach the agent's container
- The docker runtime, step by step — then submit for real

~55 min

02



From “runs on my machine” to “runs on the cluster”

Colab has no GPU. Instead of running training locally, you describe it and hand it to ClearML to execute remotely.



The old way



SSH into the box, hope a GPU is free



Fight others for VRAM, OOM crashes



No record of what ran where



The ClearML way



Submit the task to a named queue



An agent runs it on a fixed GPU slice



Live metrics + full record in the UI



```
task.execute_remotely(queue_name='gpu-18gb')
```



Three words you'll hear all hour

The whole orchestration model is just these three, working together.



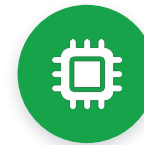
Queue

A named waiting line for tasks (e.g. gpu-18gb). You enqueue a task and walk away.



Agent

A daemon watching a queue. It pulls the next task and runs it — pulling image, code & deps for you.



Worker

The compute the agent runs on — here, a specific GPU/MIG slice. One worker = one concurrency slot.

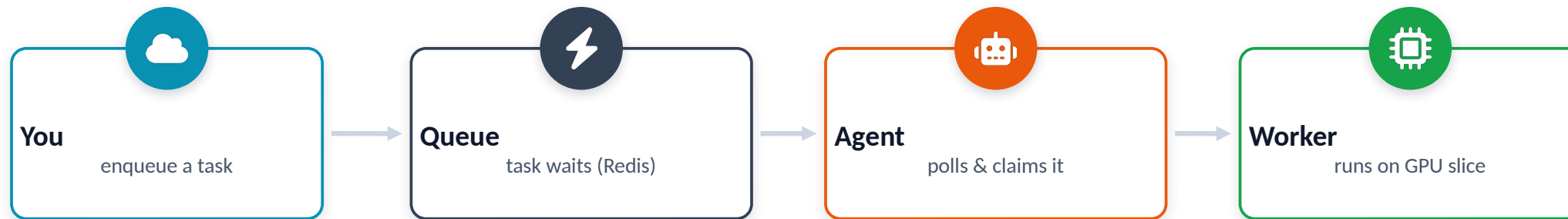
Concurrency = agents per queue. Two agents on gpu-18gb $\Rightarrow$ two jobs run there at once.



How a job actually gets run

You enqueue; the rest is automatic. The task waits in Redis until an agent is free to take it.

Enqueue → wait → poll → run



See it live: Orchestration → Workers shows every agent, what it's running, and how long. Your task moves Queued → Running → Completed in real time.



Why share one GPU at all?

An H200 has ~141 GB. ViT-Tiny needs ~2-3 GB. Handing a whole card to one tiny job wastes the box — so we slice it.



Time-slicing

soft

Jobs take turns on the whole GPU. Simple, but no memory isolation — one job can starve another.



MPS

soft

Multiple jobs run at once, sharing memory space. Faster, but a leak in one can OOM the others.



MIG

hard

The card is split into hardware-isolated instances, each with its own VRAM & compute. No cross-job OOM.

This VM uses MIG — the strongest isolation, and it works on open-source ClearML.



Soft sharing vs MIG, side by side

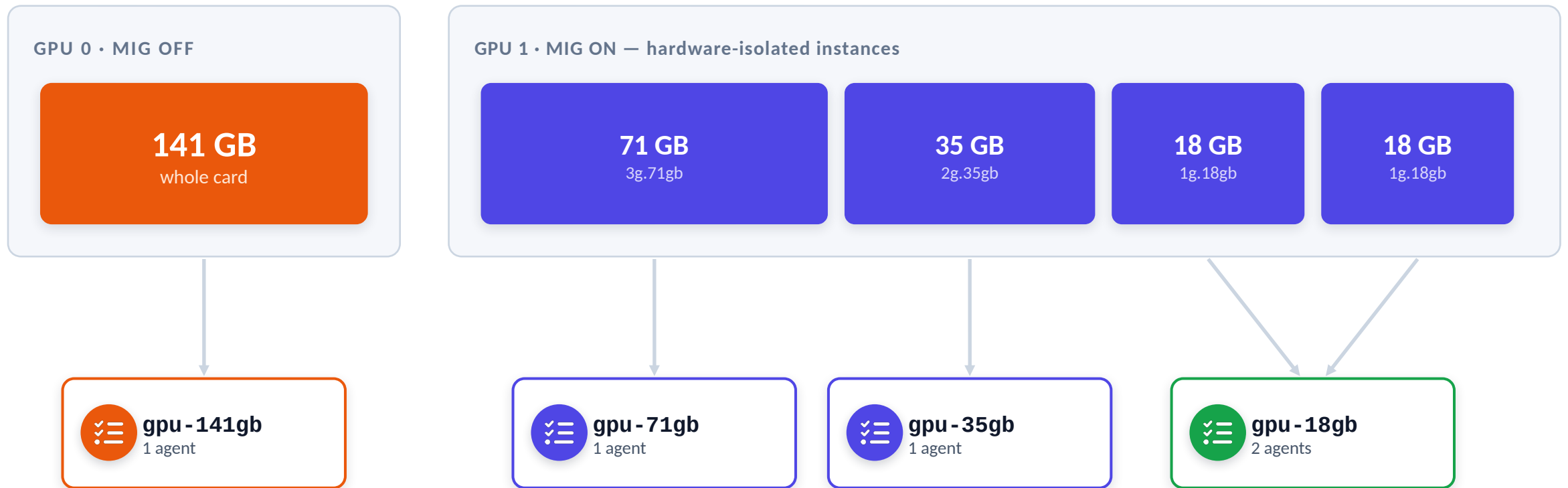
Why MIG is the right call for a shared teaching box where 30 people submit at once.

Time-slice / MPS		MIG (this VM)
Isolation	Software only	Hardware-enforced
Out-of-memory blast radius	Can take down neighbours	Contained to your own slice
VRAM guarantee	Best-effort	Fixed per instance (18/35/71 GB)
Determinism	Varies with load	Stable, predictable



Your VM's GPU fabric

Two H200s, split so the box offers both whole-card power and several isolated fractional slots at once.



Each queue maps to a fixed slice. A job that over-allocates OOMs only inside its own slice — never a neighbour's.



Pick your tier — one queue per VRAM size

Choose by how much memory the job needs. ViT-Tiny is tiny, so today everyone uses gpu-18gb.

Queue	Backing device	VRAM	Typical workload	
gpu - 18gb	GPU1 · 2×1g.18gb (2 agents)	18 GB	CV · tabular · tiny LLM	TODAY
gpu - 35gb	GPU1 · 2g.35gb (1 agent)	35 GB	small LLM: 1–7B QLoRA	
gpu - 71gb	GPU1 · 3g.71gb (1 agent)	71 GB	mid LLM: 13B QLoRA / 7B full FT	
gpu - 141gb	GPU0 · whole card (1 agent)	141 GB	largest / full-card jobs	



How your code gets to the agent

The agent runs on a different machine — so ClearML packages your run and ships it. Two ways to deliver the code.



Copy the script

Stores your entry script in the task — no git needed. But only that one file travels: no local imports or extra modules.



```
Task.force_store_standalone_script()
```



Clone the repo

Hour 2

For multi-file projects: clearml-task points the agent at a public repo; it git-clones the branch and runs from it.



```
clearml-task --repo ... --branch main
```

Either way, the task carries everything the agent needs to rebuild your run:



your code



requirements.txt



connected hyperparameters



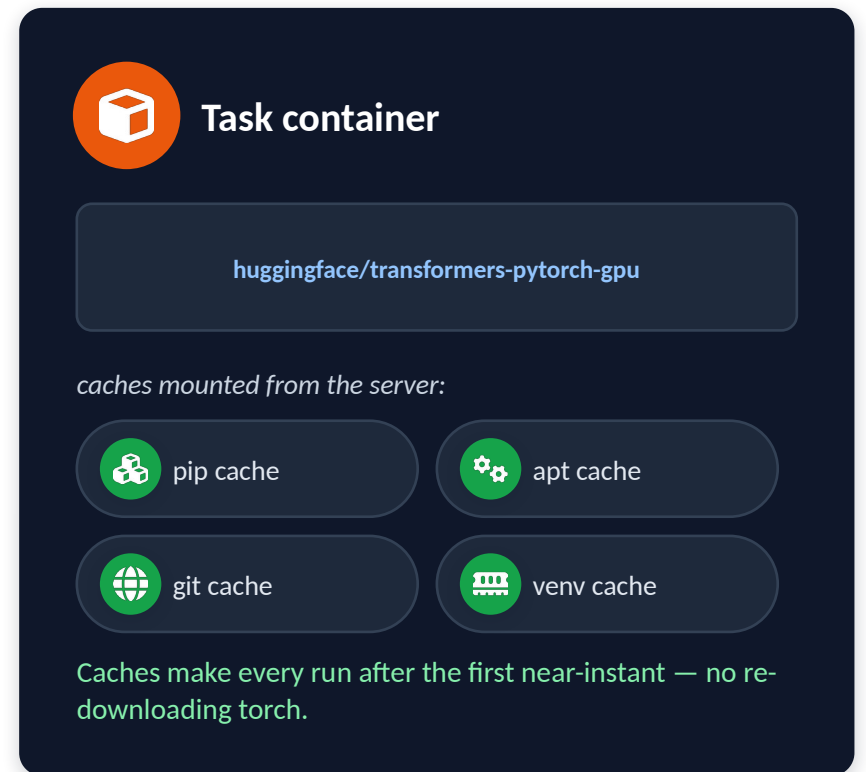
the base docker image



What the agent does, step by step

The question everyone asks: how does my script actually end up running in a GPU container? Like this.

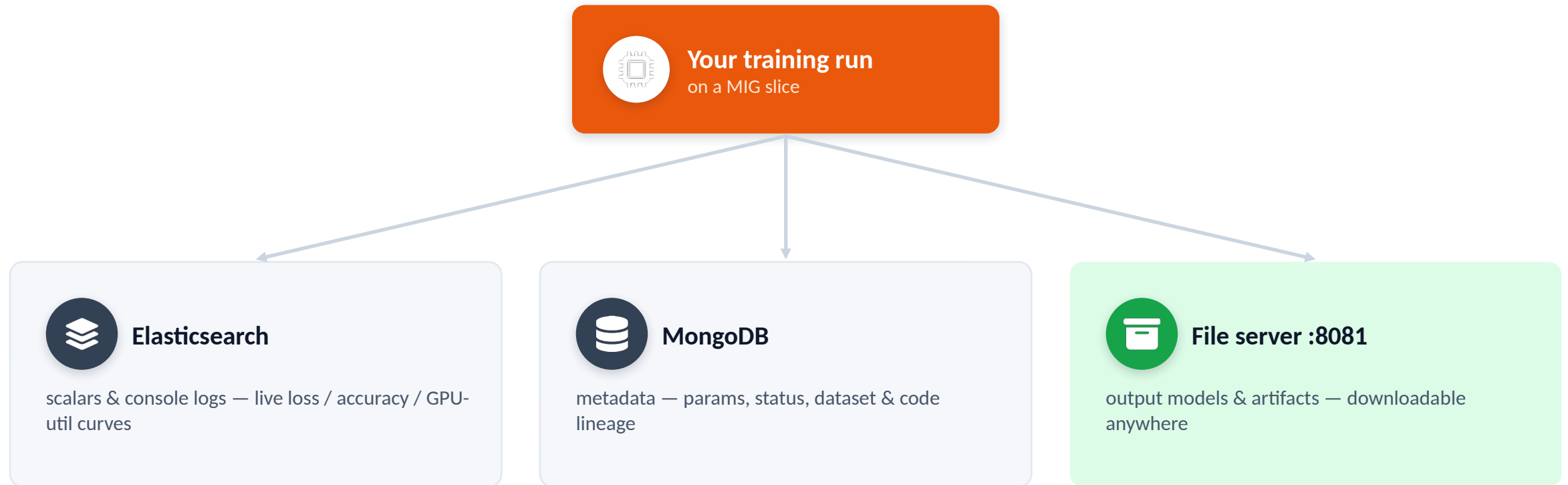
- 1 Polls the queue and claims your task
- 2 docker run the base image (HuggingFace GPU image)
- 3 Fetches the code — git clone or the stored script
- 4 Builds a venv and installs requirements.txt
- 5 Applies your hyperparameters from the task
- 6 Pins the GPU slice: `NVIDIA_VISIBLE_DEVICES=<MIG>`
- 7 Runs the script — streams logs, scalars & models back





Where your results go

As the run trains, ClearML routes each kind of output to the store built for it — and you watch it live.





HuggingFace Trainer, ClearML-aware

You already know the Trainer. One argument wires it to ClearML — the training loop is untouched.

```
hour2_finetune_vit.py

args = TrainingArguments(
    num_train_epochs=3,
    learning_rate=5e-4,
    eval_strategy='epoch',
    report_to=['clearml'],    # ← the whole trick
)
```



Scalars stream automatically

Every logged loss & eval-accuracy lands in the SCALARS tab as a live curve — no extra code.



One clean model per epoch

A callback registers a named model each epoch, so Hours 3–4 have real models to compare & publish.



You stay in control

`auto_connect_frameworks` keeps ClearML from saving every checkpoint — only the models you want.



One Colab gotcha worth knowing

Submitting from a notebook needs a specific command — this trips people up, so we call it out.



Don't: `execute_remotely()` from a cell

From a Jupyter/Colab context it mis-detects the run as a notebook and stubs the task instead of enqueueing it — so nothing reaches the queue.



Do: submit with `clearml-task`

It creates the task via the SDK and enqueues it directly. Two ways to ship the code:



Standalone — a single self-contained script — no git, but only that one file travels



From a repo (what we use) — `--repo` clones a multi-file project; needs it public + pushed to main



Demo — the whole room hits the GPU at once

1

Run the clearml-task cell

Submits ViT-Tiny training to the gpu-18gb queue.

2

Everyone submits together

30 tasks hit the same two H200s at once.

3

Open Orchestration → Workers

Watch agents pick up jobs across the MIG slices.



The big moment



Many jobs run on the same H200 — each locked to its own memory slice



Live loss & accuracy curves stream into each task's SCALARS



GPU utilization shows up per run



When it finishes, an output model registers automatically



Hour 2 — what you can now do

You've gone from a laptop notebook to real GPU training on shared, hardware-isolated hardware.



Submitted training to a shared GPU queue



Understood queues, agents, workers & MIG isolation



Learned exactly what the agent does to run your code



Watched live metrics stream from the H200 into the UI



Next: Hour 3

You'll have several runs soon. Next we clone and tweak them, compare side by side, and pick the winner — by the numbers.

[Compare runs →](#)



HOURL 3 • COMPARE

Compare

You've got a run — now make more, put them side by side, and let the numbers pick the winner.

In this hour

- Clone → tweak → re-enqueue, all in the UI
- Compare runs: curves, values, parallel coordinates
- Pick the best model programmatically by accuracy
- Model lineage — why the winner is reproducible

~40 min



Iterate without touching code

Found a run you like? Clone it, change a hyperparameter in the UI, and send the copy back to the queue.

1



Clone

Right-click your run → Clone. Creates an editable Draft.

2



Tweak

CONFIGURATION → Hyperparameters → General → edit `learning_rate` or `epochs` → Save.

3



Enqueue

Right-click the Draft → Enqueue → gpu-18gb. An agent runs the new variant.

No rebuild, no re-upload — cloning reuses the captured environment and just overrides what you changed.



Compare runs side by side

Tick two or more runs in the table, hit COMPARE, and read off exactly what changed and what it bought you.



Scalars → Graphs

Loss & accuracy curves overlaid, run against run — see which trained better, and when.



Scalars → Values

A table of last / min / max accuracy per run, side by side. The quick “who won” view.



Parallel Coordinates

Each run a line across learning_rate, epochs → accuracy. Spot the winning region at a glance.



Pick the winner in code

The UI is great for eyeballing; the SDK lets you rank every run by validation accuracy and grab the best one's ID.

```
hour3_pick_best_model.ipynb

tasks = Task.get_tasks(
    project_name=PROJECT, task_name='ViT...',
    task_filter={'status': ['completed']})

ranked = sorted(tasks, key=best_accuracy,
                reverse=True)
winner = ranked[0]
print(winner.id, winner.output_models_id)
```

SAMPLE OUTPUT

```
0.995 ViT-Tiny finetune (e83f...)
0.990 ViT-Tiny finetune (0565...)
0.980 ViT-Tiny finetune (65a6...)
```

WINNER: e83f... | accuracy 0.995

→ prints the exact publish command for Hour 4



Why the winner is reproducible

Click the winning model → LINEAGE. Everything needed to recreate it is recorded — that's what makes “the winner” meaningful.



Exact code + uncommitted changes



The full package list & versions



The dataset version it trained on



Every hyperparameter



The metrics it achieved



The container image it ran in

Reproducibility isn't a promise here — it's a recorded fact for every run.



Hour 3 — what you can now do

You can iterate fast, compare rigorously, and defend your model choice with full provenance.



Cloned and re-ran experiments without editing code



Compared runs with curves, tables & parallel coordinates



Selected the best model programmatically by accuracy



Traced a model's full lineage back to its inputs



Next: Hour 4

You've picked a winner. Next we publish it to the model registry, then pull it back by tag from a totally separate script and run inference.

[Register & reuse →](#)



HOURL 4 · REUSE

Reuse

Turn your winner into a catalogued, shareable asset — then consume it from a completely separate script.

In this hour

- The model registry — models as standalone entities
- Tag & publish — make the winner read-only
- Pull a model back by tag, no task required
- Run inference and close the loop

~45 min

04



The model registry

In ClearML, models aren't trapped inside the run that made them — they're first-class, independent objects.



Standalone entities with their own ID

A model can be fetched directly or via its creating task. It outlives the run — a true artifact entry.



Catalogued with full provenance

The registry tracks every model with lineage back to the code, data and metrics that produced it.



Browsable in the UI

The project's Model Table lists them all — filter, sort, compare, and open any model's detail page.

MODELS



ViT-Tiny — epoch 3

winner



ViT-Tiny — epoch 2



ViT-Tiny — epoch 1

Tag it, then publish it

Publishing marks the winner as the official, protected version — and it's one SDK call (or a right-click in the UI).

```
hour4_publish_and_infer.py · publish  
  
model = Model(model_id=winner_model)  
model.tags += ['winner']  
model.publish() # → read-only
```



Tag = a label you can query

“winner” lets any script find this exact model later, without knowing its ID.



Publish = read-only & protected

The model (and its creating task) become immutable — no accidental overwrites.



It now appears in the UI under Models as read-only, with a full lineage tab.



Governance at scale

The registry becomes the source of truth for “which model is approved”.



Pull it back by tag — from anywhere

The reuse half of the loop: a separate script finds the published model by its tag and runs inference. No task, no run ID.

```
hour4_publish_and_infer.py · infer

m = Model.query_models(
    tags=['winner'], only_published=True)[0]
path = InputModel(m.id).get_local_copy()

# load into ViT, classify new leaves —
# no ClearML task needed
```



Watch for



The model downloads by tag alone — the ID never appears



A predictions grid of classified leaves (DEBUG SAMPLES)



A confusion matrix + accuracy on the demo images

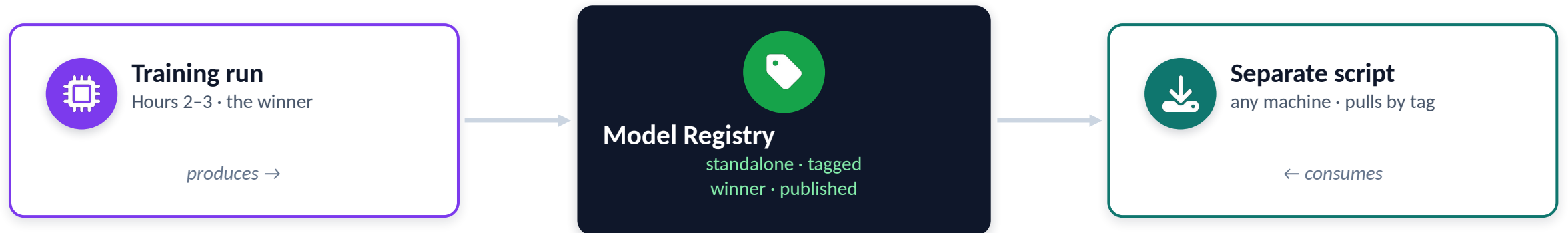


Inference runs locally on CPU — nothing sent back



The loop is closed

One run produced the model; a different script consumed it — decoupled through the registry. That's the whole point.



Neither side needs to know the other exists. The registry is the contract.



HOUR 4 • REUSE

Hour 4 — the full loop, done

You've run every stage of the MLOps loop end-to-end, on real infrastructure, from a browser.



Published the winning model to the registry with a tag



Understood models as standalone, governed entities



Pulled a model back by tag with no task or run ID



Ran inference and closed the track → reuse loop



What's next

Today you pulled a model and ran it once. In production you'd want it always-on — that's ClearML Serving, plus pipelines, HPO and more.

[Beyond the lab →](#)



From “pull and run” to “always-on”

Today's inference was a one-off, on demand. Production usually needs a model that's live behind an endpoint 24/7.



Today — local inference



You pull the model when you need it



It runs in your script/process



Perfect for batch jobs & experiments



Nothing is listening between runs



Production — ClearML Serving



A REST endpoint answers requests live



Runs in dedicated serving containers



Auto-batching, canary, monitoring



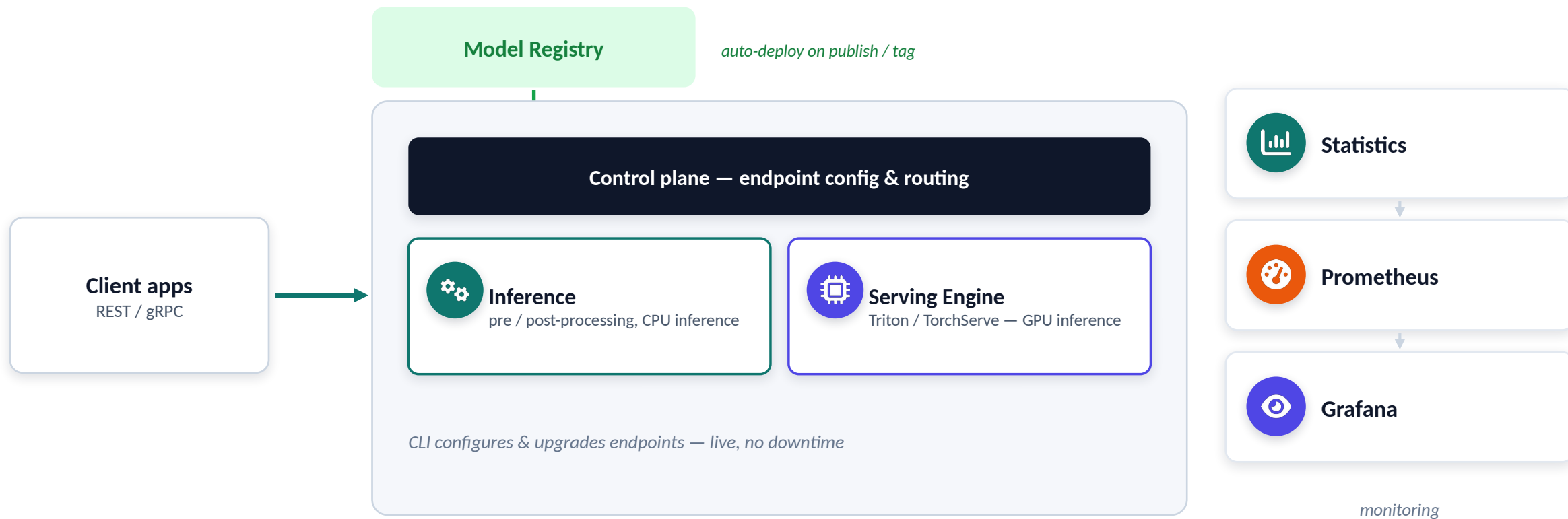
New model versions auto-deploy on publish



WHAT'S NEXT

ClearML Serving, at a glance

A CLI-driven control plane that turns a registered model into a monitored REST endpoint.





YOU RAN THE WHOLE LOOP

Track → Scale → Compare → Reuse

From a browser, on your own H200 cluster — the same platform now runs your real training, comparisons, and models.



Track



Scale



Compare



Reuse



202.131.110.56



clear.ml/docs



[the workshop repo](#)

Questions? Let's build.